

Esame Programmazione di Sistema / API Programming

Data: 2022-10-26

QUESTIONS



Domanda 1

Si illustrino le differenze tra stack e heap. Insieme alle differenze, indicare per i seguenti costrutti Rust, in modo dettagliato, dove si trovano i dati che li compongono: `Box<T>`, `RefCell<T>` e `&[T]`.

Domanda 2

Un sistema concorrente può essere implementato creando più thread nello stesso processo, creando più processi basati su un singolo thread o basati su più thread. Si mettano a confronto i corrispettivi modelli di esecuzione e si evidenzino pregi e difetti in termini di robustezza, prestazioni, scalabilità e semplicità di sviluppo di ciascuno di essi.

Domanda 3

In riferimento a programmi multi-thread, si indichi se la natura della loro esecuzione sia deterministica o meno a priori. Si produca un esempio che dimostri tale affermazione.

Domanda 4

Un componente con funzionalità di cache permette di ottimizzare il comportamento di un sistema riducendo il numero di volte in cui una funzione è invocata, tenendo traccia dei risultati da essa restituiti a fronte di un particolare dato in ingresso.

Per generalità, si assuma che la funzione accetti un dato di tipo generico K e restituisca un valore di tipo generico V .

Il componente offre un unico metodo `get(...)` che prende in ingresso due parametri, il valore k (di tipo K , clonabile) del parametro e la funzione f (di tipo $K \rightarrow V$) responsabile della sua trasformazione, e restituisce uno smart pointer clonabile al relativo valore.

Se, per una determinata chiave k , non è ancora stato calcolato il valore corrispondente, la funzione viene invocata e ne viene restituito il risultato; altrimenti viene restituito il risultato già trovato.

Il componente cache deve essere thread-safe perché due o più thread possono richiedere contemporaneamente il valore di una data chiave: quando questo avviene e il dato non è ancora presente, la chiamata alla funzione dovrà essere eseguita nel contesto di UN SOLO thread, mentre gli altri dovranno aspettare il risultato in corso di elaborazione, SENZA CONSUMARE cicli macchina.

Si implementi tale componente a scelta nei linguaggi C++ o Rust.

SOLUTIONS

Domanda 1

Differenze tra Stack e Heap:

- **Stack:** La memoria stack è gestita in modo automatico dal sistema operativo seguendo un approccio LIFO (Last-In, First-Out). L'allocazione e la deallocazione sono estremamente veloci. Tuttavia, la dimensione dei dati allocati deve essere nota a tempo di compilazione e lo spazio disponibile è solitamente limitato e prefissato. Lo stack ospita i parametri delle chiamate a funzione, gli indirizzi di ritorno e le variabili locali il cui ciclo di vita è limitato allo scope corrente.
- **Heap:** La memoria heap viene gestita dinamicamente a runtime ed è adatta per memorizzare dati la cui dimensione non è prevedibile a compile-time o può mutare. L'accesso e l'allocazione sull'heap sono più lenti per via dei meccanismi di ricerca di frammenti di memoria liberi e della gestione della frammentazione. In linguaggi come Rust, i dati in heap sono automaticamente deallocati quando i loro owner escono di scope. L'heap permette ad un dato di avere una lifetime (durata di vita) che si protrae oltre lo scope in cui viene creato.

Posizionamento dei dati per costrutti in Rust:

- **Box<T>:** Il puntatore **Box** (o smart pointer) si trova nello **stack** ed è rappresentato come un *fat pointer* che contiene sia l'indirizzo di memoria sia la lunghezza (**len**) dello slice. I dati puntati, ovvero la sequenza di elementi di tipo **T**, risiedono nello **heap**.
- **RefCell<T>:** L'intero costrutto **RefCell<T>** è allocato di default *inline* nello **stack** se dichiarato localmente (a meno che non sia contenuto dentro un altro smart pointer sull'heap come **Box** o **Rc**). Questo include il dato protetto **T** e il metadato necessario a tracciare i borrow a runtime (un contatore di **BorrowState**).
- **&[T]:** Si tratta di un riferimento ad uno slice, che si concretizza in un *fat pointer*. La sua struttura è posizionata nello **stack** (e contiene un indirizzo base e un intero per la dimensione). I dati riferiti di tipo **T** possono trovarsi **ovunque**: sullo stack (se lo slice è un sottoinsieme di un array locale), sull'heap (se si riferisce al contenuto di un **Vec** o di un **Box**), oppure nella memoria statica (come nel caso degli array immutabili di tipo **&'static [T]**).

Domanda 2

Confronto tra i modelli di esecuzione:

- **Processo Singolo con Multi-thread:**
 - *Semplicità di sviluppo:* L'implementazione è spesso più immediata per le applicazioni in cui diverse unità di calcolo devono condividere uno stato o delle variabili globali, in quanto i thread condividono il medesimo spazio d'indirizzamento (memoria, file descriptor). La complessità si sposta tuttavia sulla gestione delle race condition tramite l'uso corretto delle primitive di sincronizzazione.
 - *Prestazioni:* L'avvio di un thread è più veloce di quello di un processo e il context switch (cambio di contesto) è notevolmente più leggero, in quanto non è necessario un completo scambio della mappa di memoria virtuale o il flush della TLB (Translation Lookaside Buffer).
 - *Robustezza:* Inferiore rispetto al multi-processo. Un errore grave in un thread, come un *segmentation fault* o un panic (in modalità abort), causerà la chiusura immediata di tutti gli altri thread e quindi del processo intero.
 - *Scalabilità:* Limitata verticalmente alla macchina in uso (o ai singoli core). Inoltre, può essere ostacolata dalla contesa dei lock necessari alla sincronizzazione.
- **Multi-processo con Singolo Thread:**
 - *Semplicità di sviluppo:* Richiede una progettazione architetturale in cui i processi comunicano attraverso meccanismi di Inter-Process Communication (IPC, come pipe, socket o shared memory) che impongono serializzazione e deserializzazione (marshalling) dei dati. La sincronizzazione implicita previene le problematiche tipiche dell'accesso concorrente a locazioni di memoria.
 - *Prestazioni:* Creare nuovi processi è più gravoso in termini di tempo macchina rispetto alla creazione di thread (sebbene mitigato dall'approccio *copy-on-write* sui sistemi moderni). I cambi di contesto sono anch'essi più lenti.

- *Robustezza*: Altissima. I processi sono confinati nei loro rispettivi spazi d'indirizzamento. Se uno crasha o smette di rispondere, non corrompe né fa terminare gli altri processi del sistema.
- *Scalabilità*: Molto elevata, poiché permette anche l'estensione orizzontale su macchine distribuite o architetture di rete.

- **Multi-processo con Multi-thread:**

- Unisce i pregi e i difetti dei modelli precedenti: isolamento tra componenti applicativi distinti garantendo robustezza, accompagnato dalla possibilità di gestire molteplici connessioni e flussi all'interno della stessa task concorrente. Viene utilizzato tipicamente da grossi database server, server web, o architetture enterprise.

Domanda 3

L'esecuzione dei programmi multi-thread **non è deterministica a priori**. La sequenza esatta in cui si intervallano le istruzioni macchina dipende principalmente dalle decisioni di preemption e scheduling prese dal sistema operativo (che a loro volta dipendono dai timer e dall'attività di altri processi). Anche in presenza di un processore multi-core le latenze legate al modello gerarchico delle cache, la presenza di reordering del compilatore o di istruzioni hardware out-of-order rompono qualunque garanzia sulle sequenze di eventi in assenza di adeguati punti di barriera e sincronizzazione.

Esempio che dimostra l'affermazione (Interferenza o Data Race): Si consideri una variabile condivisa globale `a` inizializzata a 0 e due thread `T1` e `T2` che tentano, in parallelo e non sincronizzati, di eseguire la seguente istruzione apparentemente "atomica":

```
1 // Pseudocodice illustrativo
2 a = a + 1;
```

A livello hardware, questa riga di codice viene tradotta in un minimo di tre cicli istruzione, tipicamente simili a:

1. `LOAD a` in un registro interno
2. `ADD 1` al registro interno
3. `STORE registro` nella locazione di `a`

Poiché lo scheduler può interrompere in qualsiasi momento o su diversi core i tempi non sono perfettamente allineati, si potrebbe incorrere nel seguente scheduling sfavorevole:

- `T1` esegue la `LOAD` di `a` (legge 0).
- Lo scheduler ferma `T1` e fa riprendere `T2`.
- `T2` esegue la `LOAD` di `a` (legge anch'esso 0 dalla memoria o dalla cache).
- `T2` esegue l'`ADD` e la `STORE`, aggiornando la memoria in modo che valga 1.
- Lo scheduler ripristina l'esecuzione di `T1`, che ha il suo registro locale fermo a 0. Esegue l'`ADD`, producendo 1 e poi riscrive 1 su `a`.

Ci aspetteremmo che a valle dei due incrementi `a` valesse 2, mentre invece vale 1, e un terzo evento di esecuzione in ordine leggermente sfalsato produrrebbe un risultato del tutto differente.

Domanda 4

L'esercizio richiede la realizzazione di una Cache con una singola funzione `get` che accetta come secondo parametro la chiusura `f` utile a calcolare il valore da ritornare per la chiave `k`. Nel caso una chiave venga richiesta simultaneamente da più thread e non sia disponibile nel dizionario, uno solo di questi calcola `f(k)`, mentre gli altri thread vengono messi in stato di attesa tramite una **Condition Variable** (Condition Variable **Condvar**) onde evitare che vengano impiegati cicli di CPU in attesa attiva.

```

1 use std::collections::hash_map::Entry::{Occupied, Vacant};
2 use std::collections::HashMap;
3 use std::hash::Hash;
4 use std::sync::{Arc, Condvar, Mutex};
5
6 // Enum per tracciare lo stato del valore nella Hash Map.
7 enum EntryState<V> {
8     // Indica che il calcolo e' attualmente in esecuzione.
9     Pending,
10    // Indica che il risultato e' pronto, ed e' contenuto in un Arc clonabile.
11    Present(Arc<V>),
12 }
13
14 pub struct Cache<K, V>
15 where
16     K: Eq + Hash + Clone,
17 {
18     // Mappa protetta da un mutex, per permettere visibilita' in thread concorrenti.
19     m: Mutex<HashMap<K, EntryState<V>>>,
20     // Condition Variable su cui sospendere i thread in attesa del medesimo dato.
21     cv: Condvar,
22 }
23
24 impl<K, V> Cache<K, V>
25 where
26     K: Eq + Hash + Clone,
27 {
28     pub fn new() -> Self {
29         Cache {
30             m: Mutex::new(HashMap::new()),
31             cv: Condvar::new(),
32         }
33     }
34
35     pub fn get(&self, k: K, f: impl FnOnce(&K) -> V) -> Arc<V> {
36         // Prendo il possesso della lock sulla map per manipolarla.
37         let mut map = self.m.lock().unwrap();
38
39         loop {
40             // entry() effettua la lookup
41             match map.entry(k.clone()) {
42                 Occupied(e) => {
43                     match e.get() {
44                         EntryState::Pending => {
45                             // Un altro thread sta gia' calcolando la chiusura per la
46                             // chiave 'k'.
47                             // Metto in pausa il thread corrente in attesa su self.cv,
48                             // cedendo
49                             // la lock mutua (map) cosi' che altri thread possano
50                             // accedervi.
51                             // wait_while si assicura di ri-acquisire la lock solo
52                             // a fronte del cambio in Present.
53                             map = self.cv.wait_while(map, |m| {
54                                 if let Occupied(e_inner) = m.entry(k.clone()) {
55                                     if let EntryState::Present(_) = e_inner.get() {
56                                         return false; // Esci dal ciclo di attesa (wait)
57                                     }
58                                 }
59                                 true // Continua ad attendere (blocca)
60                             }).unwrap();
61                         }
62                         EntryState::Present(v) => {
63                             // Il dato era gia' calcolato e presente in map, oppure
64                             // il calcolo concorrente ha finito ed ha aggiornato la map
65                             e svegliato il thread
66                             // Siccome e' contenuto in un Arc, posso direttamente
67                             // clonare lo smart pointer e rilasciarlo
68                             return v.clone();
69                         }
70                     }
71                 }
72             }
73         }
74     }
75
76     Vacant(e) => {

```

```

68         // Siamo il PRIMO thread che richiede la risorsa k non in map.
69         // Inseriamo lo stato Pending di modo che i thread successivi
70         // si accodino alla Condvar su questa medesima map.
71         e.insert(EntryState::Pending);
72
73         // Rilascio IMMEDIATAMENTE il lock per fare in modo di:
74         // 1. permettere alle chiamate get con chiavi != k di non rimanere
in starve
75         // 2. permettere a chiamate get con k uguali di inserirsi nello
stack della condvar
76         drop(map);
77
78         // Eseguo il calcolo, potenzialmente molto oneroso. Non ho in
possesto il mutex di cache in questo punto.
79         let v = Arc::new(f(&k));
80
81         // Terminato il calcolo, ri-ottengo il mutex per rimettermi l'
ownership del value
82         // e fare in modo che possa essere usufruito ed estrapolato in
lettura.
83         map = self.m.lock().unwrap();
84         map.entry(k.clone()).and_modify(|entry| {
85             *entry = EntryState::Present(v.clone());
86         });
87
88         // Segnalo a TUTTI i thread sulla condvar di procedere a testare di
nuovo
89         self.cv.notify_all();
90
91         return v;
92     }
93 }
94 }
95 }
96 }

```